

ASG

September 11, 2026

```
[2]: import pandas as pd
import numpy as np
import os
```

```
[3]: file=pd.ExcelFile(r"C:\Users\ajint\Downloads\Airlines Use Case (1)\Airlines Use_
↳Case\UseCase - Airlines.xlsx")
print(file.sheet_names)
```

```
['flights', 'payments', 'bookings', 'passengers']
```

```
[4]: flights=pd.read_excel(file,sheet_name="flights")
payments=pd.read_excel(file,sheet_name="payments")
bookings=pd.read_excel(file,sheet_name="bookings")
passengers=pd.read_excel(file,sheet_name="passengers")
```

```
[5]: print("Flights:",flights.shape)
print("Bookings:",bookings.shape)
print("Payments:",payments.shape)
print("Passengers:",passengers.shape)
```

```
Flights: (1020, 7)
Bookings: (1000, 9)
Payments: (1000, 4)
Passengers: (1039, 9)
```

```
[6]: print("FLIGHTS",flights.columns.tolist())
print("BOOKINGS",bookings.columns.tolist())
print("PAYMENTS",payments.columns.tolist())
print("PASSENGERS",passengers.columns.tolist())
```

```
FLIGHTS ['flight_id', 'airline', 'source', 'destination', 'departure_time',
'arrival_time', 'duration']
BOOKINGS ['booking_id', 'passenger_id', 'flight_id', 'booking_date', 'status',
'passport_number', 'seat_number', 'emergency_contact_name',
'emergency_contact_phone']
PAYMENTS ['payment_id', 'booking_id', 'amount', 'payment_method']
PASSENGERS ['passenger_id', 'first_name', 'last_name', 'age', 'gender', 'email',
'phone', 'aadhaar_id', 'date_of_birth']
```

```
[7]: print("FLIGHTS:")
      print(flights.info())
      print("BOOKINGS:")
      print(bookings.info())
      print("PAYMENTS:")
      print(payments.info())
      print("PASSENGERS:")
      print(passengers.info())
```

FLIGHTS:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1020 entries, 0 to 1019

Data columns (total 7 columns):

#	Column	Non-Null Count	Dtype
0	flight_id	1020 non-null	object
1	airline	979 non-null	object
2	source	1020 non-null	object
3	destination	1020 non-null	object
4	departure_time	1020 non-null	datetime64[ns]
5	arrival_time	1020 non-null	datetime64[ns]
6	duration	1020 non-null	object

dtypes: datetime64[ns](2), object(5)

memory usage: 55.9+ KB

None

BOOKINGS:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 9 columns):

#	Column	Non-Null Count	Dtype
0	booking_id	1000 non-null	object
1	passenger_id	1000 non-null	object
2	flight_id	1000 non-null	object
3	booking_date	1000 non-null	datetime64[ns]
4	status	955 non-null	object
5	passport_number	1000 non-null	object
6	seat_number	1000 non-null	object
7	emergency_contact_name	1000 non-null	object
8	emergency_contact_phone	1000 non-null	object

dtypes: datetime64[ns](1), object(8)

memory usage: 70.4+ KB

None

PAYMENTS:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 1000 entries, 0 to 999

Data columns (total 4 columns):

#	Column	Non-Null Count	Dtype
---	--------	----------------	-------

```

---  -----  -----  -----
0  payment_id      1000 non-null  object
1  booking_id      1000 non-null  object
2  amount          952 non-null  object
3  payment_method  1000 non-null  object
dtypes: object(4)
memory usage: 31.4+ KB
None
PASSENGERS:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1039 entries, 0 to 1038
Data columns (total 9 columns):
#   Column          Non-Null Count  Dtype
---  -----  ---
0  passenger_id    1039 non-null  object
1  first_name      1039 non-null  object
2  last_name       1029 non-null  object
3  age             1039 non-null  int64
4  gender          1039 non-null  object
5  email           1039 non-null  object
6  phone           1039 non-null  object
7  aadhaar_id      1039 non-null  int64
8  date_of_birth   1039 non-null  datetime64[ns]
dtypes: datetime64[ns](1), int64(2), object(6)
memory usage: 73.2+ KB
None

```

```

[8]: print("Flight missing values")
print(flights.isnull().sum())
print("\nBookings Missing values")
print(bookings.isnull().sum())
print("\nPayments missing values")
print(payments.isnull().sum())
print("\nPassengers missing values")
print(passengers.isnull().sum())

```

```

Flight missing values
flight_id      0
airline        41
source         0
destination    0
departure_time 0
arrival_time   0
duration       0
dtype: int64

```

```

Bookings Missing values
booking_id      0

```

```

passenger_id      0
flight_id         0
booking_date      0
status            45
passport_number    0
seat_number       0
emergency_contact_name  0
emergency_contact_phone 0
dtype: int64

```

Payments missing values

```

payment_id      0
booking_id      0
amount          48
payment_method   0
dtype: int64

```

Passengers missing values

```

passenger_id      0
first_name        0
last_name         10
age              0
gender            0
email            0
phone            0
aadhaar_id        0
date_of_birth     0
dtype: int64

```

```

[9]: print("Flights duplicate rows:", flights.duplicated().sum())
      print("Bookings duplicate rows:", bookings.duplicated().sum())
      print("Payments duplicate rows:", payments.duplicated().sum())
      print("Passengers duplicate rows:", passengers.duplicated().sum())

```

```

Flights duplicate rows: 15
Bookings duplicate rows: 0
Payments duplicate rows: 0
Passengers duplicate rows: 0

```

```

[10]: duplicate_flights=flights[flights.duplicated(keep=False)].
      ↪sort_values("flight_id")
      duplicate_flights

```

```

[10]:   flight_id  airline source destination  departure_time \
550    AI020      NaN    BOM          BLR 2026-04-19 03:53:41.701
549    AI020      NaN    BOM          BLR 2026-04-19 03:53:41.701
141    AI031  Air India    DEL          MAA 2026-04-20 13:05:41.701
142    AI031  Air India    DEL          MAA 2026-04-20 13:05:41.701

```

598	AI043	Air India	CCU	DEL	2026-04-19	00:28:41.701
599	AI043	Air India	CCU	DEL	2026-04-19	00:28:41.701
603	AI070	Air India	CCU	DEL	2026-04-19	00:05:41.702
602	AI070	Air India	CCU	DEL	2026-04-19	00:05:41.702
83	AI242	UNKNOWN	BLR	CCU	2026-04-20	16:41:41.704
84	AI242	UNKNOWN	BLR	CCU	2026-04-20	16:41:41.704
940	SJ037	SpiceJet	BLR	BOM	2026-04-17	19:14:41.701
939	SJ037	SpiceJet	BLR	BOM	2026-04-17	19:14:41.701
588	SJ118	SpiceJet	DEL	BOM	2026-04-19	01:08:41.702
589	SJ118	SpiceJet	DEL	BOM	2026-04-19	01:08:41.702
106	SJ142	SpiceJet	CCU	BOM	2026-04-20	15:14:41.702
105	SJ142	SpiceJet	CCU	BOM	2026-04-20	15:14:41.702
310	SJ146	SpiceJet	HYD	BLR	2026-04-19	22:37:41.702
309	SJ146	SpiceJet	HYD	BLR	2026-04-19	22:37:41.702
606	UK013	Vistara	BOM	MAA	2026-04-18	23:52:41.701
605	UK013	Vistara	BOM	MAA	2026-04-18	23:52:41.701
935	UK049	Vistara	HYD	BOM	2026-04-17	19:47:41.701
936	UK049	Vistara	HYD	BOM	2026-04-17	19:47:41.701
758	UK139	NaN	BLR	MAA	2026-04-18	10:54:41.703
759	UK139	NaN	BLR	MAA	2026-04-18	10:54:41.703
812	UK160	Vistara	BLR	BOM	2026-04-18	06:42:41.703
813	UK160	Vistara	BLR	BOM	2026-04-18	06:42:41.703
502	UK163	Vistara	DEL	HYD	2026-04-19	07:22:41.703
503	UK163	Vistara	DEL	HYD	2026-04-19	07:22:41.703
914	UK180	Vistara	CCU	MAA	2026-04-17	21:18:41.703
915	UK180	Vistara	CCU	MAA	2026-04-17	21:18:41.703

		arrival_time	duration
550	2026-04-19	06:21:41.701	02:28:00
549	2026-04-19	06:21:41.701	02:28:00
141	2026-04-20	16:11:41.701	03:06:00
142	2026-04-20	16:11:41.701	03:06:00
598	2026-04-19	04:37:41.701	04:09:00
599	2026-04-19	04:37:41.701	04:09:00
603	2026-04-19	01:44:41.702	01:39:00
602	2026-04-19	01:44:41.702	01:39:00
83	2026-04-20	17:43:41.704	01:02:00
84	2026-04-20	17:43:41.704	01:02:00
940	2026-04-17	22:31:41.701	03:17:00
939	2026-04-17	22:31:41.701	03:17:00
588	2026-04-19	02:33:41.702	01:25:00
589	2026-04-19	02:33:41.702	01:25:00
106	2026-04-20	16:56:41.702	01:42:00
105	2026-04-20	16:56:41.702	01:42:00
310	2026-04-20	00:33:41.702	01:56:00
309	2026-04-20	00:33:41.702	01:56:00
606	2026-04-19	02:52:41.701	03:00:00

```

605 2026-04-19 02:52:41.701 03:00:00
935 2026-04-17 20:32:41.701 00:45:00
936 2026-04-17 20:32:41.701 00:45:00
758 2026-04-18 14:31:41.703 03:37:00
759 2026-04-18 14:31:41.703 03:37:00
812 2026-04-18 10:58:41.703 04:16:00
813 2026-04-18 10:58:41.703 04:16:00
502 2026-04-19 09:04:41.703 01:42:00
503 2026-04-19 09:04:41.703 01:42:00
914 2026-04-17 23:29:41.703 02:11:00
915 2026-04-17 23:29:41.703 02:11:00

```

```
[11]: print("Number of duplicate flight rows:",len(duplicate_flights))
```

```
Number of duplicate flight rows: 30
```

```
[12]: flight_id_counts = flights["flight_id"].value_counts()
      flight_id_counts[flight_id_counts>1]
```

```
[12]: flight_id
      UK049      2
      SJ118      2
      SJ142      2
      UK180      2
      UK013      2
      AI070      2
      AI043      2
      UK160      2
      SJ146      2
      AI242      2
      SJ037      2
      AI020      2
      UK139      2
      UK163      2
      AI031      2
      6F250      2
      Name: count, dtype: int64
```

```
[13]: duplicate_ids=flight_id_counts[flight_id_counts>1].index
      flights[flights["flight_id"].isin(duplicate_ids)].sort_values("flight_id")
```

```
[13]:   flight_id  airline source destination  departure_time \
253    6F250  UNKNOWN   DEL          BLR 2026-04-20 03:26:41.701
270    6F250  UNKNOWN   CCU          BLR 2026-04-20 02:23:41.702
550    AI020      NaN    BOM          BLR 2026-04-19 03:53:41.701
549    AI020      NaN    BOM          BLR 2026-04-19 03:53:41.701
141    AI031 Air India   DEL          MAA 2026-04-20 13:05:41.701
142    AI031 Air India   DEL          MAA 2026-04-20 13:05:41.701
```

599	AI043	Air India	CCU	DEL	2026-04-19	00:28:41.701
598	AI043	Air India	CCU	DEL	2026-04-19	00:28:41.701
603	AI070	Air India	CCU	DEL	2026-04-19	00:05:41.702
602	AI070	Air India	CCU	DEL	2026-04-19	00:05:41.702
83	AI242	UNKNOWN	BLR	CCU	2026-04-20	16:41:41.704
84	AI242	UNKNOWN	BLR	CCU	2026-04-20	16:41:41.704
939	SJ037	SpiceJet	BLR	BOM	2026-04-17	19:14:41.701
940	SJ037	SpiceJet	BLR	BOM	2026-04-17	19:14:41.701
588	SJ118	SpiceJet	DEL	BOM	2026-04-19	01:08:41.702
589	SJ118	SpiceJet	DEL	BOM	2026-04-19	01:08:41.702
106	SJ142	SpiceJet	CCU	BOM	2026-04-20	15:14:41.702
105	SJ142	SpiceJet	CCU	BOM	2026-04-20	15:14:41.702
310	SJ146	SpiceJet	HYD	BLR	2026-04-19	22:37:41.702
309	SJ146	SpiceJet	HYD	BLR	2026-04-19	22:37:41.702
605	UK013	Vistara	BOM	MAA	2026-04-18	23:52:41.701
606	UK013	Vistara	BOM	MAA	2026-04-18	23:52:41.701
935	UK049	Vistara	HYD	BOM	2026-04-17	19:47:41.701
936	UK049	Vistara	HYD	BOM	2026-04-17	19:47:41.701
759	UK139	NaN	BLR	MAA	2026-04-18	10:54:41.703
758	UK139	NaN	BLR	MAA	2026-04-18	10:54:41.703
812	UK160	Vistara	BLR	BOM	2026-04-18	06:42:41.703
813	UK160	Vistara	BLR	BOM	2026-04-18	06:42:41.703
503	UK163	Vistara	DEL	HYD	2026-04-19	07:22:41.703
502	UK163	Vistara	DEL	HYD	2026-04-19	07:22:41.703
914	UK180	Vistara	CCU	MAA	2026-04-17	21:18:41.703
915	UK180	Vistara	CCU	MAA	2026-04-17	21:18:41.703

		arrival_time	duration
253	2026-04-20	07:30:41.701	04:04:00
270	2026-04-20	02:56:41.702	00:33:00
550	2026-04-19	06:21:41.701	02:28:00
549	2026-04-19	06:21:41.701	02:28:00
141	2026-04-20	16:11:41.701	03:06:00
142	2026-04-20	16:11:41.701	03:06:00
599	2026-04-19	04:37:41.701	04:09:00
598	2026-04-19	04:37:41.701	04:09:00
603	2026-04-19	01:44:41.702	01:39:00
602	2026-04-19	01:44:41.702	01:39:00
83	2026-04-20	17:43:41.704	01:02:00
84	2026-04-20	17:43:41.704	01:02:00
939	2026-04-17	22:31:41.701	03:17:00
940	2026-04-17	22:31:41.701	03:17:00
588	2026-04-19	02:33:41.702	01:25:00
589	2026-04-19	02:33:41.702	01:25:00
106	2026-04-20	16:56:41.702	01:42:00
105	2026-04-20	16:56:41.702	01:42:00
310	2026-04-20	00:33:41.702	01:56:00

309	2026-04-20	00:33:41.702	01:56:00
605	2026-04-19	02:52:41.701	03:00:00
606	2026-04-19	02:52:41.701	03:00:00
935	2026-04-17	20:32:41.701	00:45:00
936	2026-04-17	20:32:41.701	00:45:00
759	2026-04-18	14:31:41.703	03:37:00
758	2026-04-18	14:31:41.703	03:37:00
812	2026-04-18	10:58:41.703	04:16:00
813	2026-04-18	10:58:41.703	04:16:00
503	2026-04-19	09:04:41.703	01:42:00
502	2026-04-19	09:04:41.703	01:42:00
914	2026-04-17	23:29:41.703	02:11:00
915	2026-04-17	23:29:41.703	02:11:00

```
[14]: print(flights["flight_id"].astype(str).sort_values().to_string(index=False))
```

```
6F001
6F002
6F003
6F004
6F005
6F006
6F007
6F008
6F009
6F010
6F011
6F012
6F013
6F014
6F015
6F016
6F017
6F018
6F019
6F020
6F021
6F022
6F023
6F024
6F025
6F026
6F027
6F028
6F029
6F030
6F031
6F032
```

6F033
6F034
6F035
6F036
6F037
6F038
6F039
6F040
6F041
6F042
6F043
6F044
6F045
6F046
6F047
6F048
6F049
6F050
6F051
6F052
6F053
6F054
6F055
6F056
6F057
6F058
6F059
6F060
6F061
6F062
6F063
6F064
6F065
6F066
6F067
6F068
6F069
6F070
6F071
6F072
6F073
6F074
6F075
6F076
6F077
6F078
6F079
6F080

6F081
6F082
6F083
6F084
6F085
6F086
6F087
6F088
6F089
6F090
6F091
6F092
6F093
6F094
6F095
6F096
6F097
6F098
6F099
6F100
6F101
6F102
6F103
6F104
6F105
6F106
6F107
6F108
6F109
6F110
6F111
6F112
6F113
6F114
6F115
6F116
6F117
6F118
6F119
6F120
6F121
6F122
6F123
6F124
6F125
6F126
6F127
6F128

6F129
6F130
6F131
6F132
6F133
6F134
6F135
6F136
6F137
6F138
6F139
6F140
6F141
6F142
6F143
6F144
6F145
6F146
6F147
6F148
6F149
6F150
6F151
6F152
6F153
6F154
6F155
6F156
6F157
6F158
6F159
6F160
6F161
6F162
6F163
6F164
6F165
6F166
6F167
6F168
6F169
6F170
6F171
6F172
6F173
6F174
6F175
6F176

6F177
6F178
6F179
6F180
6F181
6F182
6F183
6F184
6F185
6F186
6F187
6F188
6F189
6F190
6F191
6F192
6F193
6F194
6F195
6F196
6F197
6F198
6F199
6F200
6F201
6F202
6F203
6F204
6F205
6F206
6F207
6F208
6F209
6F210
6F211
6F212
6F213
6F214
6F215
6F216
6F217
6F218
6F219
6F220
6F221
6F222
6F223
6F224

6F225
6F226
6F227
6F228
6F229
6F230
6F231
6F232
6F233
6F234
6F235
6F236
6F237
6F238
6F239
6F240
6F241
6F242
6F243
6F244
6F245
6F246
6F247
6F248
6F249
6F250
6F250
6F251
6F252
6F253
6F254
6F255
6F256
6F257
6F258
6F259
6F260
6F261
6F262
6F263
6F264
6F265
6F266
6F267
6F268
6F269
6F270
6F271

6F272
AI001
AI002
AI003
AI004
AI005
AI006
AI007
AI008
AI009
AI010
AI011
AI012
AI013
AI014
AI015
AI016
AI017
AI018
AI019
AI020
AI020
AI021
AI022
AI023
AI024
AI025
AI026
AI027
AI028
AI029
AI030
AI031
AI031
AI032
AI033
AI034
AI035
AI036
AI037
AI038
AI039
AI040
AI041
AI042
AI043
AI043
AI044

AI045
AI046
AI047
AI048
AI049
AI050
AI051
AI052
AI053
AI054
AI055
AI056
AI057
AI058
AI059
AI060
AI061
AI062
AI063
AI064
AI065
AI066
AI067
AI068
AI069
AI070
AI070
AI071
AI072
AI073
AI074
AI075
AI076
AI077
AI078
AI079
AI080
AI081
AI082
AI083
AI084
AI085
AI086
AI087
AI088
AI089
AI090
AI091

AI092
AI093
AI094
AI095
AI096
AI097
AI098
AI099
AI100
AI101
AI102
AI103
AI104
AI105
AI106
AI107
AI108
AI109
AI110
AI111
AI112
AI113
AI114
AI115
AI116
AI117
AI118
AI119
AI120
AI121
AI122
AI123
AI124
AI125
AI126
AI127
AI128
AI129
AI130
AI131
AI132
AI133
AI134
AI135
AI136
AI137
AI138
AI139

AI140
AI141
AI142
AI143
AI144
AI145
AI146
AI147
AI148
AI149
AI150
AI151
AI152
AI153
AI154
AI155
AI156
AI157
AI158
AI159
AI160
AI161
AI162
AI163
AI164
AI165
AI166
AI167
AI168
AI169
AI170
AI171
AI172
AI173
AI174
AI175
AI176
AI177
AI178
AI179
AI180
AI181
AI182
AI183
AI184
AI185
AI186
AI187

AI188
AI189
AI190
AI191
AI192
AI193
AI194
AI195
AI196
AI197
AI198
AI199
AI200
AI201
AI202
AI203
AI204
AI205
AI206
AI207
AI208
AI209
AI210
AI211
AI212
AI213
AI214
AI215
AI216
AI217
AI218
AI219
AI220
AI221
AI222
AI223
AI224
AI225
AI226
AI227
AI228
AI229
AI230
AI231
AI232
AI233
AI234
AI235

AI236
AI237
AI238
AI239
AI240
AI241
AI242
AI242
AI243
AI244
AI245
AI246
AI247
AI248
AI249
AI250
AI251
AI252
AI253
AI254
AI255
SJ001
SJ002
SJ003
SJ004
SJ005
SJ006
SJ007
SJ008
SJ009
SJ010
SJ011
SJ012
SJ013
SJ014
SJ015
SJ016
SJ017
SJ018
SJ019
SJ020
SJ021
SJ022
SJ023
SJ024
SJ025
SJ026
SJ027

SJ028
SJ029
SJ030
SJ031
SJ032
SJ033
SJ034
SJ035
SJ036
SJ037
SJ037
SJ038
SJ039
SJ040
SJ041
SJ042
SJ043
SJ044
SJ045
SJ046
SJ047
SJ048
SJ049
SJ050
SJ051
SJ052
SJ053
SJ054
SJ055
SJ056
SJ057
SJ058
SJ059
SJ060
SJ061
SJ062
SJ063
SJ064
SJ065
SJ066
SJ067
SJ068
SJ069
SJ070
SJ071
SJ072
SJ073
SJ074

SJ075
SJ076
SJ077
SJ078
SJ079
SJ080
SJ081
SJ082
SJ083
SJ084
SJ085
SJ086
SJ087
SJ088
SJ089
SJ090
SJ091
SJ092
SJ093
SJ094
SJ095
SJ096
SJ097
SJ098
SJ099
SJ100
SJ101
SJ102
SJ103
SJ104
SJ105
SJ106
SJ107
SJ108
SJ109
SJ110
SJ111
SJ112
SJ113
SJ114
SJ115
SJ116
SJ117
SJ118
SJ118
SJ119
SJ120
SJ121

SJ122
SJ123
SJ124
SJ125
SJ126
SJ127
SJ128
SJ129
SJ130
SJ131
SJ132
SJ133
SJ134
SJ135
SJ136
SJ137
SJ138
SJ139
SJ140
SJ141
SJ142
SJ142
SJ143
SJ144
SJ145
SJ146
SJ146
SJ147
SJ148
SJ149
SJ150
SJ151
SJ152
SJ153
SJ154
SJ155
SJ156
SJ157
SJ158
SJ159
SJ160
SJ161
SJ162
SJ163
SJ164
SJ165
SJ166
SJ167

SJ168
SJ169
SJ170
SJ171
SJ172
SJ173
SJ174
SJ175
SJ176
SJ177
SJ178
SJ179
SJ180
SJ181
SJ182
SJ183
SJ184
SJ185
SJ186
SJ187
SJ188
SJ189
SJ190
SJ191
SJ192
SJ193
SJ194
SJ195
SJ196
SJ197
SJ198
SJ199
SJ200
SJ201
SJ202
SJ203
SJ204
SJ205
SJ206
SJ207
SJ208
SJ209
SJ210
SJ211
SJ212
SJ213
SJ214
SJ215

SJ216
SJ217
SJ218
SJ219
SJ220
SJ221
SJ222
SJ223
SJ224
SJ225
SJ226
SJ227
SJ228
SJ229
SJ230
SJ231
SJ232
SJ233
SJ234
SJ235
SJ236
SJ237
SJ238
SJ239
SJ240
SJ241
SJ242
SJ243
SJ244
SJ245
SJ246
SJ247
UK001
UK002
UK003
UK004
UK005
UK006
UK007
UK008
UK009
UK010
UK011
UK012
UK013
UK013
UK014
UK015

UK016
UK017
UK018
UK019
UK020
UK021
UK022
UK023
UK024
UK025
UK026
UK027
UK028
UK029
UK030
UK031
UK032
UK033
UK034
UK035
UK036
UK037
UK038
UK039
UK040
UK041
UK042
UK043
UK044
UK045
UK046
UK047
UK048
UK049
UK049
UK050
UK051
UK052
UK053
UK054
UK055
UK056
UK057
UK058
UK059
UK060
UK061
UK062

UK063
UK064
UK065
UK066
UK067
UK068
UK069
UK070
UK071
UK072
UK073
UK074
UK075
UK076
UK077
UK078
UK079
UK080
UK081
UK082
UK083
UK084
UK085
UK086
UK087
UK088
UK089
UK090
UK091
UK092
UK093
UK094
UK095
UK096
UK097
UK098
UK099
UK100
UK101
UK102
UK103
UK104
UK105
UK106
UK107
UK108
UK109
UK110

UK111
UK112
UK113
UK114
UK115
UK116
UK117
UK118
UK119
UK120
UK121
UK122
UK123
UK124
UK125
UK126
UK127
UK128
UK129
UK130
UK131
UK132
UK133
UK134
UK135
UK136
UK137
UK138
UK139
UK139
UK140
UK141
UK142
UK143
UK144
UK145
UK146
UK147
UK148
UK149
UK150
UK151
UK152
UK153
UK154
UK155
UK156
UK157

UK158
UK159
UK160
UK160
UK161
UK162
UK163
UK163
UK164
UK165
UK166
UK167
UK168
UK169
UK170
UK171
UK172
UK173
UK174
UK175
UK176
UK177
UK178
UK179
UK180
UK180
UK181
UK182
UK183
UK184
UK185
UK186
UK187
UK188
UK189
UK190
UK191
UK192
UK193
UK194
UK195
UK196
UK197
UK198
UK199
UK200
UK201
UK202

UK203
UK204
UK205
UK206
UK207
UK208
UK209
UK210
UK211
UK212
UK213
UK214
UK215
UK216
UK217
UK218
UK219
UK220
UK221
UK222
UK223
UK224
UK225
UK226
UK227
UK228
UK229
UK230

```
[15]: booking_flight_valid = bookings["flight_id"].astype("string").str.match(r"^(?:  
      ↳ [A-Z]{2}|\d[A-Z])\d{3}$",na=False)  
      print("Invalid booking flight IDs:", (~booking_flight_valid).sum())
```

Invalid booking flight IDs: 0

```
[16]: print(flights["airline"].value_counts(dropna=False))
```

```
airline  
IndiGo      249  
SpiceJet    240  
Air India   236  
Vistara     223  
NaN         41  
UNKNOWN     31  
Name: count, dtype: int64
```

```
[17]: # flights["airline"] = flights["airline"].astype("string").str.strip()
```

```
[18]: print(flights["airline"].value_counts(dropna=False))
```

```
airline
IndiGo      249
SpiceJet    240
Air India   236
Vistara     223
NaN         41
UNKNOWN     31
Name: count, dtype: int64
```

```
[19]: # 1. Airline code mapping
airline_code_map = {
    "UK": "Vistara",
    "AI": "Air India",
    "6F": "IndiGo",
    "SJ": "SpiceJet",
}

# 2. Clean the airline column
flights["airline"] = (
    flights["airline"]
    .astype("string")
    .str.strip()
)

# 3. Treat NaN and Unknown as missing
flights["airline"] = flights["airline"].replace(
    ["Unknown", "UNKNOWN", "unknown", ""],
    pd.NA
)

# 4. Extract the airline code from flight_id
flights["flight_code"] = (
    flights["flight_id"]
    .astype("string")
    .str.strip()
    .str[:2]
    .str.upper()
)

# 5. Identify rows where airline is missing
missing_airline = flights["airline"].isna()

# 6. Fill missing airline using flight code
flights.loc[missing_airline, "airline"] = (
    flights.loc[missing_airline, "flight_code"]
    .map(airline_code_map)
```

```
)
```

```
[20]: print(  
      flights[  
          flights["airline"].isna()  
      ][["flight_id", "flight_code", "airline"]]  
      )
```

Empty DataFrame
Columns: [flight_id, flight_code, airline]
Index: []

```
[21]: flights["source"] = (  
      flights["source"]  
      .astype("string")  
      .str.strip()  
      .str.upper()  
      )  
  
      flights["destination"] = (  
      flights["destination"]  
      .astype("string")  
      .str.strip()  
      .str.upper()  
      )
```

```
[22]: flights["source"] = (  
      flights["source"]  
      .astype("string")  
      .str.strip()  
      .str.upper()  
      )  
  
      flights["destination"] = (  
      flights["destination"]  
      .astype("string")  
      .str.strip()  
      .str.upper()  
      )
```

```
[23]: print(flights["source"].value_counts(dropna=False))  
      print()  
      print(flights["destination"].value_counts(dropna=False))
```

source
BOM 207
HYD 180
CCU 174
DEL 163

```
MAA    157
BLR    139
Name: count, dtype: Int64
```

```
destination
DEL    200
CCU    188
BOM    174
BLR    165
MAA    151
HYD    142
Name: count, dtype: Int64
```

```
[24]: flights["route"] = (flights["source"] + " → " + flights["destination"])
```

```
[25]: flights[[
        "flight_id",
        "source",
        "destination",
        "route"
    ]].head()
```

```
[25]:  flight_id source destination      route
0      SJ010    CCU          MAA  CCU → MAA
1      AI155    BOM          CCU  BOM → CCU
2      UK094    BOM          CCU  BOM → CCU
3      AI245    BOM          CCU  BOM → CCU
4      AI192    MAA          BOM  MAA → BOM
```

```
[26]: flights["departure_time"] = pd.to_datetime(
        flights["departure_time"],
        format="%d-%m-%Y %H:%M:%S",
        errors="coerce")

flights["arrival_time"] = pd.to_datetime(
        flights["arrival_time"],
        format="%d-%m-%Y %H:%M:%S",
        errors="coerce"
    )
```

```
[29]: print(flights[[
        "departure_time",
        "arrival_time"
    ]].dtypes)
```

```
departure_time    datetime64[ns]
arrival_time      datetime64[ns]
dtype: object
```

```
[30]: print("Missing departure times:",
        flights["departure_time"].isna().sum())

print("Missing arrival times:",
        flights["arrival_time"].isna().sum())
```

Missing departure times: 0

Missing arrival times: 0

```
[31]: flights["calculated_duration"] = (
        flights["arrival_time"] -
        flights["departure_time"]
    )
```

```
[32]: flights[[
        "flight_id",
        "departure_time",
        "arrival_time",
        "calculated_duration"
    ]].head(10)
```

```
[32]:  flight_id      departure_time      arrival_time \
0    SJ010 2026-04-20 23:38:41.701 2026-04-21 02:32:41.701
1    AI155 2026-04-20 23:35:41.703 2026-04-21 01:23:41.703
2    UK094 2026-04-20 23:26:41.702 2026-04-21 01:11:41.702
3    AI245 2026-04-20 23:07:41.704 2026-04-21 01:43:41.704
4    AI192 2026-04-20 23:05:41.703 2026-04-21 04:04:41.703
5    SJ158 2026-04-20 23:05:41.703 2026-04-21 01:24:41.703
6    6F196 2026-04-20 23:04:41.703 2026-04-21 00:47:41.703
7    AI080 2026-04-20 23:03:41.702 2026-04-21 00:35:41.702
8    6F025 2026-04-20 23:02:41.701 2026-04-20 23:57:41.701
9    6F251 2026-04-20 22:56:41.703 2026-04-20 23:37:41.703
```

```
    calculated_duration
0    0 days 02:54:00
1    0 days 01:48:00
2    0 days 01:45:00
3    0 days 02:36:00
4    0 days 04:59:00
5    0 days 02:19:00
6    0 days 01:43:00
7    0 days 01:32:00
8    0 days 00:55:00
9    0 days 00:41:00
```

```
[33]: flights["duration_minutes"] = (
        flights["calculated_duration"]
        .dt.total_seconds() / 60
    )
```

```
)
```

```
[34]: flights[[
        "flight_id",
        "duration_minutes"
    ]].head()
```

```
[34]:   flight_id  duration_minutes
0      SJ010             174.0
1      AI155             108.0
2      UK094             105.0
3      AI245             156.0
4      AI192             299.0
```

```
[35]: print(flights["duration_minutes"].describe())
```

```
count      1020.000000
mean        162.919739
std          87.335793
min         -1140.000000
25%          99.000000
50%         165.000000
75%         232.000000
max          300.000000
Name: duration_minutes, dtype: float64
```

```
[36]: flights[
        (flights["duration_minutes"] <= 0) |
        (flights["duration_minutes"] > 24 * 60)
    ][[
        "flight_id",
        "source",
        "destination",
        "departure_time",
        "arrival_time",
        "duration_minutes"
    ]]
```

```
[36]:   flight_id source destination  departure_time  arrival_time \
355      SJ192   HYD          BOM 2026-04-19 18:45:42 2026-04-18 23:45:42

        duration_minutes
355                 -1140.0
```

```
[37]: import datetime
import numpy as np

# 1. Ensure departure and arrival are converted to datetime
```

```

flights['departure_time'] = pd.to_datetime(flights['departure_time'],
    ↪errors='coerce')
flights['arrival_time'] = pd.to_datetime(flights['arrival_time'],
    ↪errors='coerce')

# 2. Calculate duration from timestamps (handling overnight/cross-day flights)
def calculate_duration(row):
    dep = row['departure_time']
    arr = row['arrival_time']
    if pd.isnull(dep) or pd.isnull(arr):
        return np.nan
    # If arrival is earlier than departure, it's an overnight flight
    if arr < dep:
        arr += pd.Timedelta(days=1)
    return round((arr - dep).total_seconds() / 60, 2)

flights['calculated_duration_mins'] = flights.apply(calculate_duration, axis=1)

# 3. Robust parser for the original 'duration' column
def parse_original_duration(val):
    if pd.isnull(val):
        return np.nan
    if isinstance(val, datetime.time):
        return val.hour * 60 + val.minute + val.second / 60
    if isinstance(val, datetime.timedelta):
        return val.total_seconds() / 60
    if isinstance(val, (int, float)):
        return float(val)

    val_str = str(val).strip()
    try:
        parts = val_str.split(':')
        if len(parts) == 3:
            return int(parts[0]) * 60 + int(parts[1]) + float(parts[2]) / 60
        elif len(parts) == 2:
            return int(parts[0]) * 60 + int(parts[1])
    except Exception:
        pass

    return pd.to_numeric(val, errors='coerce')

flights["duration_minutes_original"] = flights["duration"].
    ↪apply(parse_original_duration)

# 4. Compare both side-by-side cleanly
comparison_df = flights[['flight_id', 'departure_time', 'arrival_time',
    ↪'duration_minutes_original', 'calculated_duration_mins']]

```

```
print(comparison_df.head())
```

	flight_id	departure_time	arrival_time	\
0	SJ010	2026-04-20 23:38:41.701	2026-04-21 02:32:41.701	
1	AI155	2026-04-20 23:35:41.703	2026-04-21 01:23:41.703	
2	UK094	2026-04-20 23:26:41.702	2026-04-21 01:11:41.702	
3	AI245	2026-04-20 23:07:41.704	2026-04-21 01:43:41.704	
4	AI192	2026-04-20 23:05:41.703	2026-04-21 04:04:41.703	

	duration_minutes_original	calculated_duration_mins
0	174.0	174.0
1	108.0	108.0
2	105.0	105.0
3	156.0	156.0
4	299.0	299.0

```
[38]: # Calculate the absolute difference between original and calculated durations
flights["duration_difference"] = flights["duration_minutes_original"] -
    flights["calculated_duration_mins"]

# Filter rows where the difference is greater than 0.01 minutes (to handle
    floating-point precision)
duration_errors = flights[
    flights["duration_difference"].abs() > 0.01
]
print("Duration inconsistencies:", len(duration_errors))
```

Duration inconsistencies: 0

```
[39]: duration_errors[[
    "flight_id",
    "departure_time",
    "arrival_time",
    "duration",
    "duration_minutes_original",
    "duration_minutes"
]]
```

```
[39]: Empty DataFrame
Columns: [flight_id, departure_time, arrival_time, duration,
duration_minutes_original, duration_minutes]
Index: []
```

```
[40]: flights["overnight_flight"] = (
    flights["arrival_time"].dt.date >
    flights["departure_time"].dt.date
)
```

```
[41]: print(  
      flights["overnight_flight"].value_counts()  
    )
```

```
overnight_flight  
False      896  
True       124  
Name: count, dtype: int64
```

```
[42]: flights["departure_date"] = (  
      flights["departure_time"].dt.date  
    )
```

```
[43]: flights["departure_hour"] = (  
      flights["departure_time"].dt.hour  
    )
```

```
[44]: flights["data_quality_status"] = "Valid"
```

```
[45]: flights["duration_minutes"] = pd.to_numeric(  
      flights["duration_minutes"],  
      errors="coerce"  
    )
```

```
[46]: invalid_duration = (  
      flights["duration_minutes"] <= 0  
    ) | (  
      flights["duration_minutes"] > 24 * 60  
    )  
  
flights.loc[  
    invalid_duration,  
    "data_quality_status"  
] = "Invalid Duration"
```

```
[47]: flights["anomaly_flag"] = (  
      flights["data_quality_status"] != "Valid"  
    )
```

```
[48]: flights["anomaly_flag"].value_counts()
```

```
[48]: anomaly_flag  
False      1019  
True         1  
Name: count, dtype: int64
```

```
[49]: bookings = bookings.copy()
```

```
[50]: text_columns = [
        "booking_id",
        "passenger_id",
        "flight_id",
        "status",
        "passport_number",
        "seat_number",
        "emergency_contact_name",
        "emergency_contact_phone"
    ]

    for col in text_columns:
        bookings[col] = (
            bookings[col]
            .astype("string")
            .str.strip()
        )
```

```
[51]: print(
        bookings["status"]
        .value_counts(dropna=False)
    )
```

```
status
CONFIRMED    320
CANCELLED    314
PENDING      291
<NA>         45
INVALID       30
Name: count, dtype: Int64
```

```
[52]: bookings["status"] = bookings["status"].fillna("UNKNOWN")
```

```
[53]: print(bookings["status"].value_counts())
```

```
status
CONFIRMED    320
CANCELLED    314
PENDING      291
UNKNOWN       45
INVALID       30
Name: count, dtype: Int64
```

```
[54]: booking_id_valid = bookings["booking_id"].str.match(
        r"^B\d+$",
        na=False
    )
```

```
print("Invalid booking IDs:",
      (~booking_id_valid).sum())
```

Invalid booking IDs: 0

```
[55]: passenger_id_valid = bookings["passenger_id"].str.match(
      r"^P\d+$",
      na=False
    )

print(
    "Invalid passenger IDs:",
    (~passenger_id_valid).sum()
)
```

Invalid passenger IDs: 0

```
[56]: booking_flight_valid = bookings["flight_id"].astype("string").str.match(
      r"^(?:[A-Z]{2}|\d[A-Z])\d{3}$",
      na=False
    )

print(
    "Invalid booking flight IDs:",
    (~booking_flight_valid).sum()
)
```

Invalid booking flight IDs: 0

```
[57]: bookings["booking_date"] = pd.to_datetime(
      bookings["booking_date"],
      errors="coerce"
    )
```

```
[58]: print(bookings["booking_date"].isna().sum())
```

0

```
[59]: passengers_clean = passengers.copy()
```

```
[60]: passengers_clean["aadhaar_masked"] = (
    passengers_clean["aadhaar_id"]
    .astype("string")
    .str[-4:]
    .radd("*****")
)
```

```
[61]: passengers_clean["phone_masked"] = (
    passengers_clean["phone"]
```

```
.astype("string")
.str[-4:]
.radd("*****")
)
```

```
[62]: passengers_analytics = passengers_clean[
    [
        "passenger_id",
        "age",
        "gender",
        "date_of_birth",
        "aadhaar_masked",
        "phone_masked"
    ]
].copy()
```

```
[63]: passengers_analytics["gender"] = (
    passengers_analytics["gender"]
    .astype("string")
    .str.strip()
    .str.upper()
)
```

```
[64]: payments = payments.copy()
```

```
[65]: payments["payment_id"] = (
    payments["payment_id"]
    .astype("string")
    .str.strip()
)

payments["booking_id"] = (
    payments["booking_id"]
    .astype("string")
    .str.strip()
)
```

```
[66]: print(payments["amount"].dtype)
```

object

```
[67]: payments["amount"] = pd.to_numeric(
    payments["amount"],
    errors="coerce"
)
```

```
[68]: print(payments["amount"].dtype)
```

float64

```
[69]: print(
      "Missing payment amounts:",
      payments["amount"].isna().sum()
    )
```

Missing payment amounts: 78

```
[70]: print(
      payments["payment_method"]
      .value_counts(dropna=False)
    )
```

payment_method
UPI 358
CARD 329
NETBANKING 313
Name: count, dtype: int64

```
[71]: payments["payment_method"] = (
      payments["payment_method"]
      .astype("string")
      .str.strip()
      .str.upper()
    )
```

```
[72]: print(
      payments["payment_method"]
      .value_counts(dropna=False)
    )
```

payment_method
UPI 358
CARD 329
NETBANKING 313
Name: count, dtype: Int64

```
[73]: invalid_payment_bookings = payments[
      ~payments["booking_id"].isin(
          bookings["booking_id"]
      )
    ]

    print(
      "Payments with invalid booking IDs:",
      len(invalid_payment_bookings)
    )
```

Payments with invalid booking IDs: 0

```
[74]: invalid_booking_flights = bookings[
        ~bookings["flight_id"].isin(
            flights["flight_id"]
        )
    ]

print(
    "Bookings with invalid flight IDs:",
    len(invalid_booking_flights)
)
```

Bookings with invalid flight IDs: 0

```
[75]: invalid_booking_passengers = bookings[
        ~bookings["passenger_id"].isin(
            passengers["passenger_id"]
        )
    ]

print(
    "Bookings with invalid passenger IDs:",
    len(invalid_booking_passengers)
)
```

Bookings with invalid passenger IDs: 0

```
[76]: quality_summary = pd.DataFrame({
    "Dataset": [
        "Flights",
        "Bookings",
        "Payments",
        "Passengers"
    ],
    "Rows": [
        len(flights),
        len(bookings),
        len(payments),
        len(passengers)
    ],
    "Duplicate_Rows": [
        flights.duplicated().sum(),
        bookings.duplicated().sum(),
        payments.duplicated().sum(),
        passengers.duplicated().sum()
    ],
    "Missing_Values": [
        flights.isna().sum().sum(),
        bookings.isna().sum().sum(),

```

```

        payments.isna().sum().sum(),
        passengers.isna().sum().sum()
    ]
})

quality_summary

```

```

[76]:      Dataset  Rows  Duplicate_Rows  Missing_Values
0     Flights  1020             15             2
1    Bookings  1000              0             0
2    Payments  1000              0            78
3  Passengers  1039              0            10

```

```

[77]: print("==== MISSING VALUES BY COLUMN =====")

```

```

print("\nFLIGHTS")
print(flights.isna().sum())

print("\nBOOKINGS")
print(bookings.isna().sum())

print("\nPAYMENTS")
print(payments.isna().sum())

print("\nPASSENGERS")
print(passengers.isna().sum())

```

```

===== MISSING VALUES BY COLUMN =====

```

```

FLIGHTS
flight_id           0
airline             0
source             0
destination         0
departure_time      0
arrival_time        0
duration            0
flight_code         0
route              0
calculated_duration 0
duration_minutes    0
calculated_duration_mins 0
duration_minutes_original 1
duration_difference 1
overnight_flight    0
departure_date      0
departure_hour      0
data_quality_status 0

```

```
anomaly_flag          0
dtype: int64
```

BOOKINGS

```
booking_id            0
passenger_id          0
flight_id             0
booking_date          0
status               0
passport_number       0
seat_number           0
emergency_contact_name 0
emergency_contact_phone 0
dtype: int64
```

PAYMENTS

```
payment_id            0
booking_id            0
amount                78
payment_method         0
dtype: int64
```

PASSENGERS

```
passenger_id          0
first_name             0
last_name             10
age                   0
gender                0
email                 0
phone                 0
aadhaar_id            0
date_of_birth          0
dtype: int64
```

```
[78]: flights_analytics = flights[
    [
        "flight_id",
        "airline",
        "source",
        "destination",
        "route",
        "departure_time",
        "arrival_time",
        "departure_date",
        "departure_hour",
        "duration_minutes",
        "overnight_flight",
```

```

        "anomaly_flag",
        "data_quality_status"
    ]
].copy()

```

```
[79]: flights_analytics.head()
```

```
[79]:
```

	flight_id	airline	source	destination	route	departure_time	\
0	SJ010	SpiceJet	CCU	MAA	CCU → MAA	2026-04-20 23:38:41.701	
1	AI155	Air India	BOM	CCU	BOM → CCU	2026-04-20 23:35:41.703	
2	UK094	Vistara	BOM	CCU	BOM → CCU	2026-04-20 23:26:41.702	
3	AI245	Air India	BOM	CCU	BOM → CCU	2026-04-20 23:07:41.704	
4	AI192	Air India	MAA	BOM	MAA → BOM	2026-04-20 23:05:41.703	

		arrival_time	departure_date	departure_hour	duration_minutes	\
0	2026-04-21	02:32:41.701	2026-04-20	23	174.0	
1	2026-04-21	01:23:41.703	2026-04-20	23	108.0	
2	2026-04-21	01:11:41.702	2026-04-20	23	105.0	
3	2026-04-21	01:43:41.704	2026-04-20	23	156.0	
4	2026-04-21	04:04:41.703	2026-04-20	23	299.0	

	overnight_flight	anomaly_flag	data_quality_status
0	True	False	Valid
1	True	False	Valid
2	True	False	Valid
3	True	False	Valid
4	True	False	Valid

```
[80]: flights_analytics = flights_analytics.drop_duplicates()
```

```
[81]: print(
    "Remaining duplicate rows:",
    flights_analytics.duplicated().sum()
)
```

Remaining duplicate rows: 0

```
[82]: bookings_analytics = bookings[
    [
        "booking_id",
        "passenger_id",
        "flight_id",
        "booking_date",
        "status",
        "seat_number"
    ]
].copy()
```

```
[83]: payments_analytics = payments[
    [
        "payment_id",
        "booking_id",
        "amount",
        "payment_method"
    ]
].copy()
```

```
[84]: print("Flights Analytics:", flights_analytics.shape)
print("Bookings Analytics:", bookings_analytics.shape)
print("Payments Analytics:", payments_analytics.shape)
print("Passengers Analytics:", passengers_analytics.shape)
```

```
Flights Analytics: (1005, 13)
Bookings Analytics: (1000, 6)
Payments Analytics: (1000, 4)
Passengers Analytics: (1039, 6)
```

```
[85]: print("==== FINAL VALIDATION =====")
print("Flights duplicate rows:", flights_analytics.duplicated().sum())
print("Bookings duplicate rows:", bookings_analytics.duplicated().sum())
print("Payments duplicate rows:", payments_analytics.duplicated().sum())
print("Passengers duplicate rows:", passengers_analytics.duplicated().sum())
print("Flights missing values:", flights_analytics.isna().sum().sum())
print("Bookings missing values:", bookings_analytics.isna().sum().sum())
print("Payments missing values:", payments_analytics.isna().sum().sum())
print("Passengers missing values:", passengers_analytics.isna().sum().sum())
```

```
===== FINAL VALIDATION =====
Flights duplicate rows: 0
Bookings duplicate rows: 0
Payments duplicate rows: 0
Passengers duplicate rows: 0
Flights missing values: 0
Bookings missing values: 0
Payments missing values: 78
Passengers missing values: 0
```

```
[86]: output_folder = r"C:\Users\ajint\Downloads\ASG_Airlines_Output"
os.makedirs(output_folder, exist_ok=True)
```

```
[87]: flights_analytics.to_csv(os.path.join(output_folder, "flights_clean.
    ↳ csv"), index=False)
bookings_analytics.to_csv(os.path.join(output_folder, "bookings_clean.
    ↳ csv"), index=False)
payments_analytics.to_csv(os.path.join(output_folder, "payments_clean.
    ↳ csv"), index=False)
```

```
passengers_analytics.to_csv(os.path.join(output_folder, "passengers_clean.  
↪csv"), index=False)
```

[]:

[]:

[]:

[]: